

The Rising CPU:GPU Ratio in AI Infrastructure: Drivers, Trends, and Implications

AI infrastructure is shifting as growth in inference workload and reinforcement learning drive higher CPU:GPU ratios, increasing the importance of CPU capacity for orchestration, simulation, and efficient utilization of large GPU clusters

Authors

Ram Varra

Senior Principal Engineer

Sachin Ashtikar

Senior Principal Engineer

Vipul Lal

Senior Principal Engineer

Shesha Krishnapura

Intel Fellow and Intel IT CTO

Contents

Executive Summary.....	1
AI Inference Growth Is Driving Higher CPU:GPU Ratios	2
Reinforcement Learning (RL) Is Driving the Growth of CPUs in AI Infrastructure	4
Conclusion.....	6
Appendix – References	7

Executive Summary

This paper explores two primary market trends and workload architectures that are driving increased demand for CPUs, relative to GPUs, for AI implementations:

- **Accelerated AI inferencing workload growth.** A higher CPU:GPU ratio is required for inferencing workloads compared to training workloads—especially as agentic AI becomes prevalent across multiple industries.
- **The renaissance of reinforcement learning (RL).** RL has matured from a video game niche to complex 3D simulation environments (such as robotics, precision medicine, and algorithmic trading) that require high numbers of CPUs.

AI training is dominated by GPU compute, where large datasets are fed into GPUs for dense linear algebra compute operations. In contrast, large-scale inference systems tend to be CPU-heavy because of orchestration, data handling, and business logic—all of which take up a higher percentage of CPU resources compared to many relatively lighter GPU inference calls. Larger GPU clusters require larger CPU infrastructure to manage interconnect traffic and distributed task scheduling, and to avoid GPU underutilization (expensive waste). Cloud providers add more CPUs per GPU to keep GPUs saturated.

Additionally, AI industry forecasts indicate that training growth is flattening, while AI inference is expected to grow much faster due to the adoption of agentic AI systems. The combination of these factors is driving higher CPU:GPU ratios in AI infrastructure implementations for both enterprises and cloud providers.

Due to advances in autonomous and semi-autonomous systems (robots, self-driving vehicles, smart grids, and adaptive networks), several industries are adopting RL. RL training requires rich simulation environments that are primarily run on CPUs. This paper covers RL architecture and key industry use cases and associated workloads. The paper also discusses RL from Human Feedback (RLHF)—a Generative AI-centric use case of RL that aligns AI models with human preferences—which makes their outputs more helpful, safe, and consistent with human values.

AI Inference Growth Is Driving Higher CPU:GPU Ratios

As model sizes increase and models run across multi-GPU clusters, CPU-bound components must manage communications, memory paging, interconnect orchestration, and distributed scheduling. VM-based deployments often suffer latency from poor CPU–GPU coordination, further motivating cloud platforms to deploy more CPUs to improve distributed pipeline performance. Next-gen GPU architectures from NVIDIA, AMD, and Intel depend even more on CPU–GPU fabric coordination for optimal performance. Higher CPU:GPU ratios help reduce network and scheduling bottlenecks in distributed AI systems.



Higher CPU:GPU ratios help reduce network and scheduling bottlenecks in distributed AI systems.

Enterprises are gradually shifting from using AI models that simply “answer questions” to AI agents that “complete tasks.” Deloitte reports that the AI agent market may expand as more enterprises prepare to orchestrate agents,¹ which will further drive the need for a higher CPU:GPU ratio in AI clusters due to increased orchestration complexity, heterogeneous tool use, and sequential logic.

Inference Workloads Are Growing Faster than Training

Enterprise dollars increasingly flow to inference as firms mature from experimentation (training) to deployment (inference). Inference now accounts for the majority of AI workloads:

- Lenovo’s CEO, Yuanqing Yang, predicted at CES 2026 that the historical 80% training/20% inference split is reversing to 80% inference/20% training.²
- Deloitte estimates that inference workloads comprised half of all AI compute in 2025 and will rise to account for two-thirds of all AI compute in 2026,³ while enterprises are increasing production inference rather than just model development, with model API spending increasing from USD 3.5B to USD 8.4B in 2025.⁴
- The Futurum Group predicts that inference workloads (such as copilots, agents, and real-time assistants) will drive more revenue than training by the end of 2026.⁵
- McKinsey reports that AI compute is shifting toward a high-availability, inference-heavy future.⁶

CPU Bottlenecks Limit GPU Utilization

Modern AI systems depend on CPUs to handle data loading, preprocessing, scheduling, memory paging, batching, and inference routing. If the CPU cannot keep up, GPUs sit idle, wasting extremely expensive GPU resources.

CPUs act as the “air-traffic controller,” ensuring GPUs stay busy by managing data movement, batching inference requests, and offloading memory operations. If CPUs lag, GPU efficiency drops and inference time rises.

These CPU-bound orchestration tasks become even more significant as AI workloads shift from training to high-volume inference and agentic AI at a global scale.

Comparing CPU Needs in Inferencing Versus Training

Table 1 shows that CPUs play a relatively smaller role in training, compared to inferencing—underscoring the need for a higher CPU:GPU ratio for inferencing workloads.

Table 1. CPU Roles for Training and Inferencing

Dimension	Training	Inferencing
Primary Bottleneck	GPUs/accelerators for forward and backward passes (CPU is a secondary bottleneck)	CPU-side request handling and pre-/post processing for lighter GPU/CPU inference kernels
Typical CPU:GPU Ratio Trend	Lower CPU:GPU ratio Cluster optimized for GPU density and interconnect	Higher CPU:GPU ratio Many services and threads per accelerator, plus CPU-only inference paths
Key CPU Tasks	Data loading, augmentation, orchestration, logging, and distributed training control	API logic, routing, batching, tokenization, feature transforms, ranking, and formatting responses
Latency Versus Throughput Priority	Maximize throughput and time-to-train Latency is less critical	Tight per-request latency service-level agreements (SLAs) and high queries per second (QPS) CPU must absorb traffic spikes

Inferencing Workload Characteristics that Drive CPU Affinity

When a user submits an inference request, it first lands at the inference API server, which queues and forwards it to the runtime engine—a critical component running on the CPU. The runtime engine performs multiple optimization tasks to keep the GPU fully utilized and minimize latency. These tasks include batching, key-value cache (KV cache) paging, and graph orchestration. Once the data is prepped and optimized, it’s sent to the GPU, where inference is executed. After processing, the CPU finalizes and returns the results to the user.

This entire pipeline depends on the host CPU’s ability to handle multiple simultaneous AI queries without creating bottlenecks. If the CPU isn’t fast enough, latency spikes, GPU efficiency drops, and response times suffer—leading to wasted compute resources.

During inference, GPU work per request is relatively small (especially with optimized kernels, quantization, and other efficiency techniques) but CPU tasks (token streaming, caching, prompt assembly, retrieval, and routing) dominate latency budgets. Data pipelines (tokenization, vector search, feature extraction, and filtering) often consume more time than the GPU forward pass. Industry guidance shows CPU orchestration is now a stronger determinant of inference throughput than raw GPU FLOPs in many cases. For example, benchmarks from AMD show that high-frequency CPUs improve GPU inference throughput by up to 9% (even with high-end industry workhorse GPUs like NVIDIA H100 or AMD MI300), demonstrating that CPU-side throughput is a limiting factor.⁷

In summary, overall AI system performance is limited by how quickly CPUs can feed, coordinate, and schedule tasks.

Table 2 shows the common tasks required for inference and their CPU affinities. Due to these factors, cloud providers are scaling CPUs to meet the inference explosion—they realize that the agentic AI era requires massive CPU power for the orchestration, data retrieval, and logic that surrounds every inference call.

Table 2. CPU Roles for Training and Inferencing^A

Pipeline Stage	Typical Operations	CPU-Bound (%)
Data Ingestion	Read from storage, decompression, and stream buffering	85-95%
Cleaning	Filtering, deduplication, and validation	90-100%
Transformation	Resizing, normalization, and tokenization	70-85%
Augmentation	Random crops, rotations, and text perturbations	60-80%
Batching	Collation, padding, and dimensional alignment	95-100%
Format Conversion	PyTorch/TensorFlow tensor creation	80-90%

^A Based on expertise and experiences with working on real-world production AI pipelines and general industry knowledge.

Multi-Tenant GPU Clusters Require Heavy CPU-Side Resources

Large clouds run many customers' workloads on shared-GPU clusters. This requires:

- Queue management
- Request deduplication
- Security isolation
- Resource scheduling
- Multi-instance GPU (MIG) or slice allocation for GPU sharing

All these tasks are primarily CPU-driven operations. The overhead of these tasks grows in proportion to GPU density: Higher GPU density requires more CPUs for management overhead.

AI Workloads Are Becoming More Multi-Stage and CPU-Heavy

AI applications—especially retrieval, agents, and pipelines—increasingly include chains of operations that run before and after GPU compute. Modern AI services now often involve retrieval-augmented generation (RAG), embeddings search, agentic chains, multi-step workflows, and orchestration layers, all of which rely heavily on CPU-side logic. Analysis of cloud providers' workloads indicates that multi-stage pipelines have variable, bursty, latency-sensitive CPU loads, unlike traditional stable workloads.

Complex agentic AI workloads often require repeated cycles of planning, tool use, reflection, and refinement to complete tasks, which can drive reliance on large frontier

models and GPU-intensive inference. However, agentic approaches that break complex problems into smaller, structured subtasks can reduce dependence on large-parameter models and shift more of the workload to CPUs. In many cases, using a large language model (LLM) to generate and execute code in a sandbox—an operation largely handled by CPUs—has proven more effective for solving agentic tasks. Figure 1 shows how complex AI workloads benefit from leveraging code generation and execution versus relying on reasoning of text/JSON inputs for complex tasks.⁸

Figure 2 illustrates how complex AI workloads are orchestrated through multiple specialized agents that plan, coordinate tools, generate and/or execute code, and iteratively refine results.⁹ Such an architecture increases CPU-oriented orchestration alongside accelerator-based model inference.

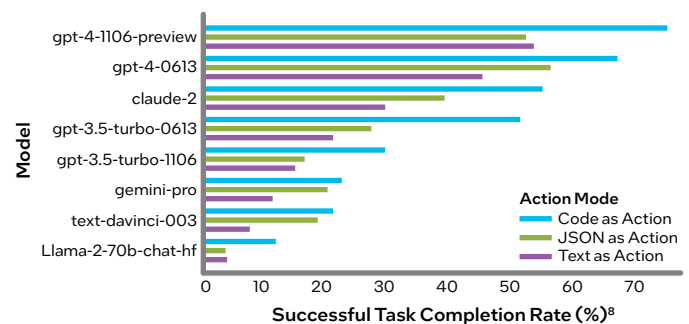


Figure 1. Agentic task advantages through structured planning and code execution.⁸

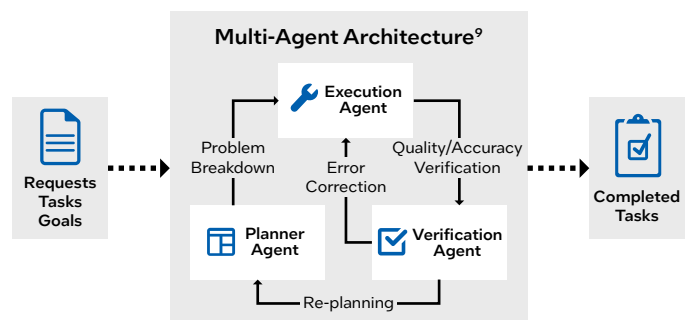


Figure 2. An increased CPU:GPU ratio can benefit multi-agent architectures, where execution agents use VMs as sandboxes to execute code.⁹

Table 3 on the following page illustrates the capability and computational differences between code-generation and complex-planning LLM inferencing workloads. As the implementation of agentic AI flows leverages more code execution patterns, the need for complex large parameter models reduces, and CPU requirements for code execution sandboxes grow. This is a contributing factor in the increasing CPU:GPU ratio growth trend in AI inference infrastructure.

Table 3. Comparison of Code-Generation and Complex-Planning LLM Computational Differences

	Code-Generation LLMs	Complex Planning LLMs
Computational Requirements	- Code generation involves deterministic patterns (syntax completion, small logical reasoning, and local context with short-range attention). - Inference is shallow, single, forward, one-pass generation. - Performance can improve by running smaller models multiple times under a fixed compute budget—e.g., repeated 7B/13B inference can match or surpass 70B generation under equal FLOP budgets. - Code generation scales horizontally on low/mid-range GPU infrastructure (multiple small runs) better than general reasoning can scale.	- Planning models require recursive reasoning, multi-step chain-of-thought reasoning, self-reflection loops, and descriptions of other tools/agents. - Inference requires multiple forward passes and larger attention operations across longer reasoning traces. - Compute demand grows with long context windows (200K–1M tokens range in frontier models) and dependency chains, making inference substantially more expensive. - Complex-Planning LLMs need multiple forward passes + longer context = 5–50× more compute per query compared to straightforward code generation.
Typical Model Size (Number of Parameters)	7B–34B	70B–1T+ (frontier)
Memory Requirements	7–24 GB	40–160 GB+/multi-GPU
Inference Cost	Single pass, moderate	Multi-pass, very high
Context Window	16K–128K tokens	200K–1M+ tokens
Hardware	Low/mid-range GPUs	High-end data center GPUs

Networking, Memory, and Distributed Coordination Require More CPUs

Hyperscale AI architecture involves tens of thousands of GPUs in cluster configurations. Bigger GPU clusters require proportionally larger CPU infrastructure to manage interconnect traffic and distributed task scheduling. Here are just a few reasons CPUs play a vital role:

- **CPUs for scale.** Cluster orchestration stacks (Kubernetes control plane, Slurm, etc.) use CPUs to allocate GPUs to workloads, share GPUs between jobs, manage GPU memory and utilization, and schedule GPU tasks across nodes.
- **CPUs for GPU orchestration.** MIT Technology Review highlights hyperscale data centers where “hundreds of thousands of specialized GPUs are connected by fiber optic nervous systems”¹⁰ and require significant coordination and data throughput, which is accomplished via CPUs.
- **CPU performance for high throughput and low latency.** Networking stacks (InfiniBand, Ethernet, and network interface cards) and distributed training/inference schedulers depend heavily on CPUs.
- **CPUs for workload orchestration and graph scheduling.** These tasks—which include deciding where and when workloads run (which nodes or clusters), allocating resources, and managing dependencies between tasks—are recognized as CPU-heavy layers required for efficient AI compute.

Energy, Power, and Cooling Constraints Push Data Centers to Optimize with CPUs

AI data centers are reaching physical limits:

- AI facilities consume massive power—some reaching gigawatt-scale, equivalent to entire cities.
- U.S. AI data centers consumed 176 TWh in 2024, and are projected to reach up to 580 TWh by 2028.¹¹ This forces better GPU utilization via CPU-side efficiency.

Scaling CPUs is often more energy-efficient than scaling GPUs, thereby improving overall power utilization. Because GPUs are so energy-intensive, data centers use CPUs to:

- Offload ancillary tasks.
- Reduce GPU idle time.
- Improve energy-per-token economics.

Reinforcement Learning (RL) Is Driving the Growth of CPUs in AI Infrastructure

AI labs such as DeepMind, OpenAI, Tesla, NVIDIA Research, Google Brain, Meta FAIR, and leading robotics/autonomy groups are focusing on RL in areas where sequential decision making, simulation, and closed-loop optimization provide clear advantages. As shown in Figure 3,¹² RL and RL with Human Feedback (RLHF) require a combination of environment stepping, control logic, search, and trajectory management—all traditionally executed on CPUs as well as CPU-first tasks that scale with the number of environments and the algorithmic control flow.

 Richer environments require more CPU cycles for simulation and data movement.

Richer environments (such as robotics physics, multi-agent driving, and high-fidelity sensors) require more CPU cycles for simulation and data movement. In addition, RLHF adoption needs longer pipelines and distributed scheduling. CPU resources manage reward evaluation, sampling, and GPU cluster orchestration.

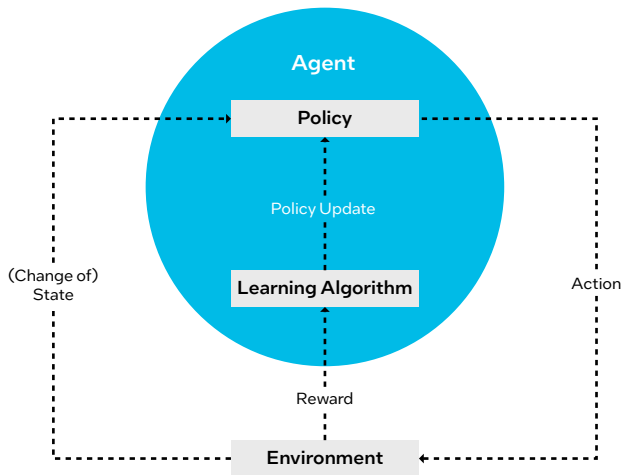


Figure 3. Reinforcement learning (RL) framework.¹²

RL Industry Use Cases and Examples

Table 4 provides an overview of RL industry use cases with specific examples of applications.

Table 4. Use Cases and Examples of Reinforcement Learning (RL)

Industry Use Case	Examples
Autonomous Vehicles and Robotics Control Lane changes, collision avoidance, robotic grasping, locomotion, and policy transfer from simulation to the real world	- Real-time vehicle decision-making (Tesla Autopilot) - High-fidelity robotic control (like locomotion and dexterous hand manipulation) via GPU-accelerated simulators like Isaac Gym and Brax, enabling millions of physics steps per second - Autonomous driving frameworks in simulators such as CARLA
Large-Scale Game AI and Strategic Reasoning Strategic planning tasks that generalize to real-world domains such as robotics and agent systems	- AlphaGo/AlphaZero/AlphaStar, OpenAI Five, and other large-scale self-play systems - Multi-agent RL for competitive and cooperative games - Model-based and tree search-driven RL systems for deep strategic reasoning
Industrial Automation and Process Optimization Industrial optimization, supply chain optimization, and manufacturing control	- Supply chain routing - Factory process control - Energy grid stability and load balancing - Chemical/industrial sequence optimization
Autonomous Decision Making in Agentic AI Systems Move from static LLMs to agent workflows, with RL as a natural foundation for agentic decision policies	- Self-improving AI agents that plan, reflect, and refine behavior - Multi-step planning and tool use decision frameworks - RL-trained policy heads for agent orchestration
Reinforcement Learning with Human Feedback (RLHF) LLM alignment of foundational and frontier models	- Safety-aligned LLM policy shaping - Preference-based reward modelling - Multi-turn dialogue optimization - Red-teaming and behavioral correction - Standard practice in Generative AI labs (OpenAI, Anthropic, Google DeepMind)
Finance, Trading, and Portfolio Optimization Sequential decision optimization in high-frequency environments	- Algorithmic trading - Portfolio rebalancing - Risk management and hedging policies - Market simulations

RL Training Architectures Need CPUs

As teams shift workloads from purely supervised training to RL, the number of CPU environment workers (and their orchestration cost) grows. RL’s algorithmic machinery incorporates non-GPU control logic, tree search, and actor-side preprocessing, which are CPU-heavy tasks that scale with environment complexity and the number of parallel rollouts. Figure 4¹³ compares the classical actor-critic RL architecture adapted for Brain-Machine Interface to a fully connected feed-forward neural network. In architecture A, the actor maps commands into actions and the critic gives evaluation feedback using the environment. In architecture B, the critic is used to adjust the model weights using evaluative feedback.

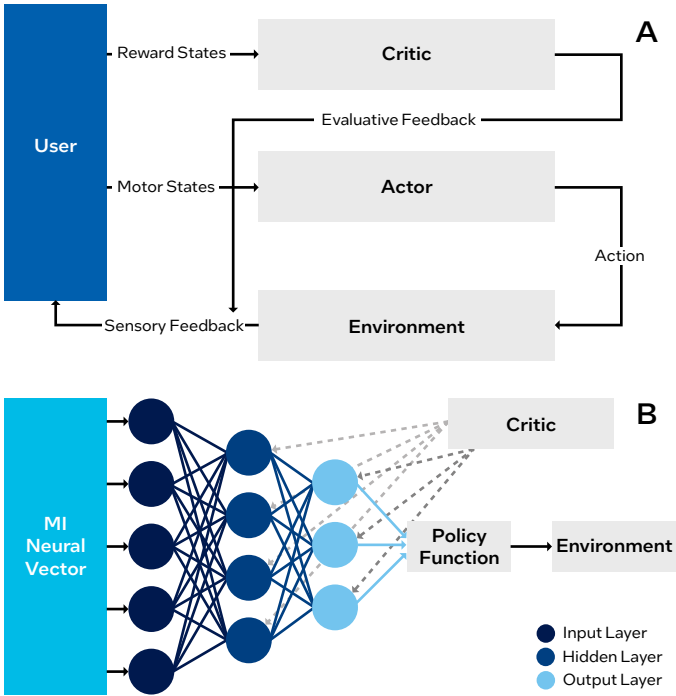


Figure 4. Architecture of actor-critic reinforcement learning (RL).¹³

Consider the following examples of the close relationship between RL architecture and CPUs:

- Modern distributed RL agents separate actors (environment stepping and inference calls) from learners (optimizer updates), and in many widely used systems, the actors run on CPUs. Importance Weighted Actor-Learner Architecture (IMPALA) uses an actor–learner design, which scales to thousands of machines by decoupling acting from learning; actors collect experience on CPUs while a central learner trains on accelerators.
- Ray RLlib, a mainstream production RL framework, exposes scaling via EnvRunner actors and explicitly assigns CPU resources per EnvRunner (and typically many environments per runner), confirming that sample collection is often parallelized across CPU processes.
- Model-based RL and planning-heavy paradigms (such as the AlphaZero/MuZero families) integrate Monte Carlo Tree Search (MCTS) and environment evaluations inside the training loop. Implementations and follow-on work underscore that training requires massive parallel rollouts and search, often executed on CPU actors, thereby increasing CPU demand.
- Even in model-free RL, off-policy corrections (such as IMPALA’s V-trace) and experience queueing/prioritization add CPU-side compute in actor/collector threads, replay buffers, and preprocessing stages.

RL's Increasing Environment Complexity Requires Larger CPU Resources

As RL environments grow in fidelity (such as in physics applications) and realism (in multi-sensor and multi-agent systems), CPU cycles for simulation, I/O, and orchestration rise steeply, increasing CPU needs in RL training clusters. Here are some specific examples of how environmental complexity drives the need for more CPU resources:

- RL for robotics and dexterous manipulation uses high-fidelity physics, contact dynamics, and large observation spaces. While the physics and tensor computation are offloaded to GPUs, CPUs are critical to feeding the execution pipes to maintain high throughput.
- Autonomous driving simulators (such as CARLA) introduce multi-agent scenes, traffic, LiDAR/RGB sensor pipelines, and scenario logic. CPUs are required for implementing simulation, processing perception modalities, and controlling abstractions used in RL studies.
- In any RL system, the amount of RAM and the number of CPU cores gate how many environments can be spun up. In large-scale Proximal Policy Optimization (PPO) implementations, CPUs often dominate the rollout rate (actors), while GPUs primarily accelerate gradient updates.

Conclusion

Modern AI stacks are no longer GPU-only. At production scale, CPUs can significantly influence end-to-end throughput by orchestrating data flows, executing environment logic, scheduling distributed jobs, and running agentic toolchains that surround accelerator kernels. Two industry trends are especially relevant to the increasingly important role of CPUs in AI clusters:

- Inference is overtaking training in compute share and spend, shifting previously steady GPU-dominated growth toward CPUs for orchestration, data processing, routing, batching, KV cache management, retrieval, code execution, and tool use.
- The growth of agentic AI and the increasing use of RL and RLHF amplify these CPU responsibilities and are reshaping capacity planning.

Overall, this paper highlights the role of CPUs as a strategic control plane for AI systems. Future AI infrastructure planning must explicitly model CPU growth as a first-class driver of cost, performance, and energy-per-token economics, rather than assuming GPUs alone determine infrastructure efficiency.

Appendix – References

- IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures - <https://arxiv.org/abs/1802.01561>
- Importance-weighted Actor-Learner Architecture - <https://tkukurin.github.io/impala>
- Massively Scaling Reinforcement Learning with SEED RL - <https://research.google/blog/massively-scaling-reinforcement-learning-with-seed-rl/>
- Why GPU cloud is replacing traditional VM-based compute for AI workloads - <https://www.gmicloud.ai/blog/why-gpu-cloud-is-replacing-traditional-vm-based-compute-for-ai-workloads>
- The next big shifts in AI workloads and hyperscaler strategies - <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-next-big-shifts-in-ai-workloads-and-hyperscaler-strategies>
- Tech Trends 2026: The AI infrastructure reckoning: Optimizing compute strategy in the age of inference economics - <https://www.deloitte.com/us/en/insights/topics/technology-management/tech-trends/2026/ai-infrastructure-compute-strategy.html>
- Unlocking exponential value with AI agent orchestration - <https://www.deloitte.com/us/en/insights/industry/technology/technology-media-and-telecom-predictions/2026/ai-agent-orchestration.html>
- The LLM Inference Wars: A Strategic Analysis of CPU, GPU, and Custom Silicon - <https://uplatz.com/blog/the-llm-inference-wars-a-strategic-analysis-of-cpu-gpu-and-custom-silicon/>
- GPU inference vs. CPU inference: Speed, cost and scalability - <https://www.gmicloud.ai/blog/gpu-inference-vs-cpu-inference-speed-cost-and-scalability>
- AI Inference Acceleration on CPUs - <https://www.intel.com/content/www/us/en/developer/articles/technical/ai-inference-acceleration-on-intel-cpus.html>
- Understanding Performance Implications of LLM Inference on CPUs - https://seonjinna.github.io/assets/pdf/iiswc24_CPULLM.pdf
- 2025 Mid-Year LLM Market Update: Foundation Model Landscape + Economics - https://menlovc.com/wp-content/uploads/2025/11/menlo_ventures_mid-year_llm_report-2025.pdf
- Why Your Host CPU Matters More Than You Think - Maximizing GPU Efficiency - <https://www.amd.com/en/blogs/2025/why-your-host-cpu-matters-more-than-you-think-ma.html>
- Characterizing and Optimizing LLM Inference Workloads on CPU-GPU Coupled Architectures - <https://arxiv.org/pdf/2504.11750>
- Hyperscale AI data centers - <https://www.technologyreview.com/2026/01/12/1129982/hyperscale-ai-data-centers-energy-usage-2026-breakthrough-technology/>
- LLM Model Size: Comparison Chart & Performance Guide - <https://labeledyourdata.com/articles/llm-fine-tuning/llm-model-size>
- Planning with code execution - <https://learn.deeplearning.ai/courses/agent-ai/lesson/egr0a8/planning-with-code-execution>
- The Larger the Better? Improved LLM Code-Generation via Budget Reallocation - <https://arxiv.org/pdf/2404.00725>
- From Mind to Machine: The Rise of Manus AI as a Fully Autonomous Digital Agent - <https://arxiv.org/pdf/2505.02024>
- coding-llm-benchmarks-guide - <https://github.com/olekkut/coding-llm-benchmarks-guide>
- Scaling AI Compute in 2025: Balancing Massive Power Demands with Sustainable Infrastructure - <https://ai2.work/technology/ai-tech-computational-power-needs-2025/>



¹ Deloitte, November 2025, "Unlocking exponential value with AI agent orchestration."

² Computer World, January 2026, "CES 2026: AI compute sees a shift from training to inference."

³ Ibid.

⁴ Menlo Ventures, July 2025, "2025 Mid-Year LLM Market Update: Foundation Model Landscape + Economics."

⁵ Futurum, December 2025, "2H 2025 AI Platforms Market Sizing and Five Year Forecast."

⁶ McKinsey and Company, February 2026, "The future of AI workloads."

⁷ AMD.com, March 2025, "Why Your Host CPU Matters More Than You Think - Maximizing GPU Efficiency."

⁸ Figure 1 is adapted from "Executable Code Actions Elicit Better LLM Agents" by Xingyao Wang et al, used under CC BY 4.0.

⁹ Figure 2 is adapted from "From Mind to Machine: The Rise of Manus AI as a Fully Autonomous Digital Agent" by Minjie Shen et al, used under CC BY 4.0.

¹⁰ MIT Technology Review, January 2026, "Hyperscale AI data centers."

¹¹ U.S. Department of Energy, December 2024, "DOE Releases New Report Evaluating Increase in Electricity Demand from Data Centers."

¹² Figure 3 is adapted from the following source: Nikolopoulou, K. (August 15, 2023). Easy Introduction to Reinforcement Learning. Scribbr. Retrieved March 13, 2026, from <https://www.scribbr.com/ai-tools/reinforcement-learning/>

¹³ Figure 4 is adapted from "A confidence metric for using neurobiological feedback in actor-critic reinforcement learning based brain-machine interfaces" by Noeline W. Prins et al, used under CC BY 4.0. Originally published in Frontiers in Neuroscience, May 25, 2014, Section Neuroprosthetics, Volume 8 - 2014 | <https://doi.org/10.3389/fnins.2014.00111>.